WPF

C# поддерживает различные технологии для разработки пользовательских интерфейсов (UI).

Windows Presentation Foundation (WPF)

WPF (Windows Presentation Foundation) — это современная технология Microsoft для создания десктопных приложений на Windows, которая является частью экосистемы .NET и представляет собой продвинутую подсистему для построения графических интерфейсов. Она поддерживает мощные возможности векторной графики, гибкую стилизацию через XAML и революционную систему привязки данных.

Преимущества:

– Отделение логики от представления через паттерн MVVM (Model-View-ViewModel), что упрощает поддержку и тестирование приложений.

– Поддержка сложной графики, анимации и эффектов.

– Широкие возможности кастомизации интерфейса, включая использование XAML (eXtensible Application Markup Language) для разметки.

Недостатки:

– Более сложная кривая обучения для разработчиков, особенно для тех, кто привык работать с WinForms.

– Может потребовать больше ресурсов для рендеринга сложных интерфейсов.

XAML (eXtensible Application Markup Language) — язык разметки, используемый для инициализации объектов в технологиях на платформе .NET. Применительно к WPF (а также к Silverlight) данный язык используется прежде всего для создания пользовательского интерфейса декларативным путем. То есть его область шире.

XAML предлагает очень простую и ясную схему определения различных элементов и их свойств. Каждый элемент, как и любой элемент XML, должен иметь открытый и закрытый тег.

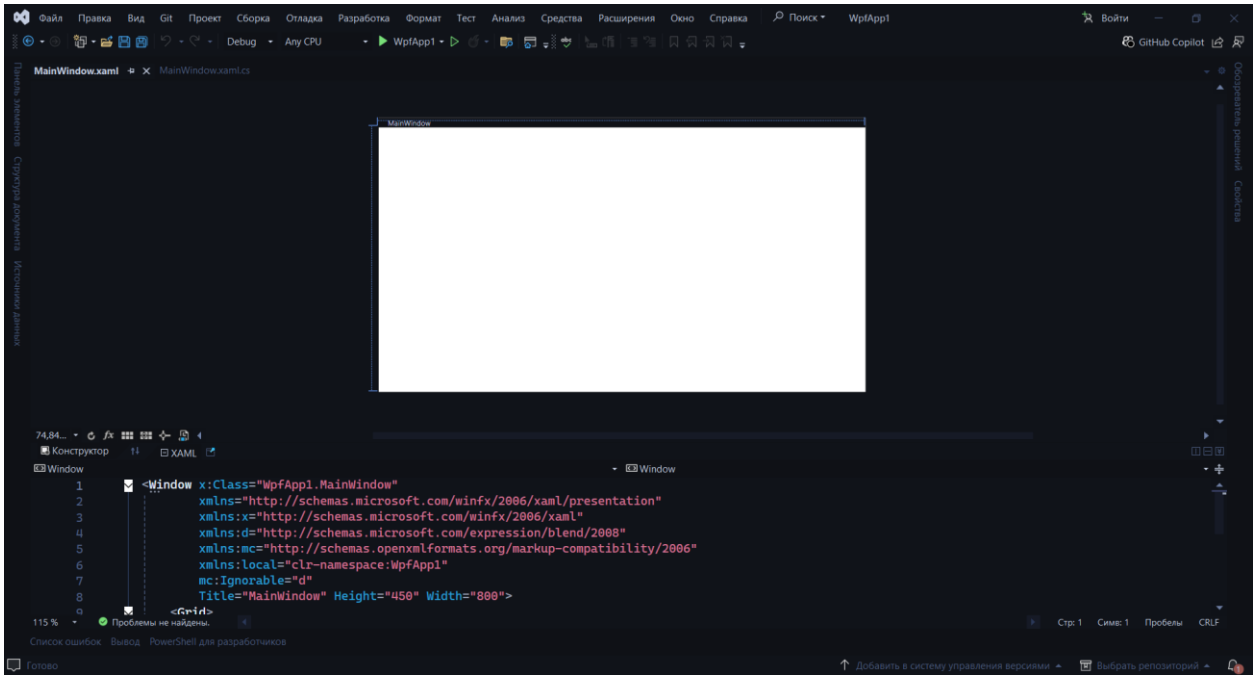
XAML — не является обязательной частью приложения, мы вобще можем обходиться без него, создавая все элементы в файле связанного с ним кода на языке C#. Однако использование XAML все-таки несет некоторые преимущества:

– Возможность отделить графический интерфейс от логики приложения, благодаря чему над разными частями приложения могут относительно автономно работать разные специалисты: над интерфейсом - дизайнеры, над кодом логики - программисты.

– Компактность, понятность, код на XAML относительно легко поддерживать.

СОЗДАНИЕ ПРОЕКТА

1. Открыть Visual Studio (**НЕ CODE**)
2. Создание проекта
3. Приложение WPF (Майкрософт)
4. Указать имя и расположение проекта
5. !!!Платформу оставить .Net 8.0



Рабочая область

При создании нового проекта WPF он уже содержит файлы с кодом xaml. Так, создаваемый по умолчанию в проекте файл MainWindow.xaml.

Начальная структура разметки уже содержит несколько строк, это подключение пространств, которые не нужно удалять или изменять. При необходимости можно подключать новые пространства.

Также присутствует разметка окна с контейнером Grid

```
    Title="MainWindow" Height="450" Width="800"> // название нашего окна и его размер
  <Grid>
    // сетка окна, здесь размещаются элементы интерфейса
  </Grid>
</Window>
```

XAML предлагает очень простую и ясную схему определения различных элементов и их свойств. Каждый элемент, как и любой элемент XML, должен иметь открытый и закрытый тег, как в случае с элементом Window:

```
<Window></Window>
```

Либо элемент может иметь сокращенно форму с закрывающим слешем в конце, наподобие:

```
<Window />
```

Также нашим главным элементом будет «Обозреватель решений». В нем находятся все файлы проекта, помимо главного окна в нем также есть файлы:

– App.config — это текстовый файл в формате XML, который используется в приложениях, разработанных на платформе .NET Framework, для хранения настроек конфигурации.

– App.xaml — это декларативная стартовая точка приложения. Он автоматически создаётся Visual Studio при создании нового приложения WPF. Здесь также будут прописываться стили для элементов интерфейса.

Работа с графической часть производится в файлах с расширением .xaml, весь бэк прописывается в файлах .xaml.cs

ЭЛЕМЕНТЫ РАЗМЕТКИ

Компоновка

Компоновка представляет собой размещение элементов в контейнере. Благодаря компоновке можно расположить и настроить расположение элементов интерфейса и реализовать их адаптивность. В WPF компоновка осуществляется при помощи специальных контейнеров. Фреймворк предоставляет нам следующие контейнеры: Grid, UniformGrid, StackPanel, WrapPanel, DockPanel и Canvas.

В WPF при компоновке и расположении элементов внутри окна нам надо придерживаться следующих принципов:

– Нежелательно указывать явные размеры элементов (за исключеним минимальных и максимальных размеров). Размеры должны определяться контейнерами.

– Нежелательно указывать явные позицию и координаты элементов внутри окна. Позиционирование элементов всецело должно быть прерогативой контейнеров. И контейнер сам должен определять, как элемент будет располагаться. Если нам надо создать сложную систему компоновки, то мы можем вкладывать один контейнер в другой, чтобы добиться максимально удобного расположения элементов управления.

Окна — первые контейнеры, с которыми вы встретитесь при разработке.

Проще говоря, это самостоятельное окно операционной системы Windows, которое пользователь может:

- перемещать
- сворачивать
- разворачивать
- закрывать

- изменять размер

Каждое WPF-приложение обычно начинается с главного окна — MainWindow.

Важные особенности окна

Окно:

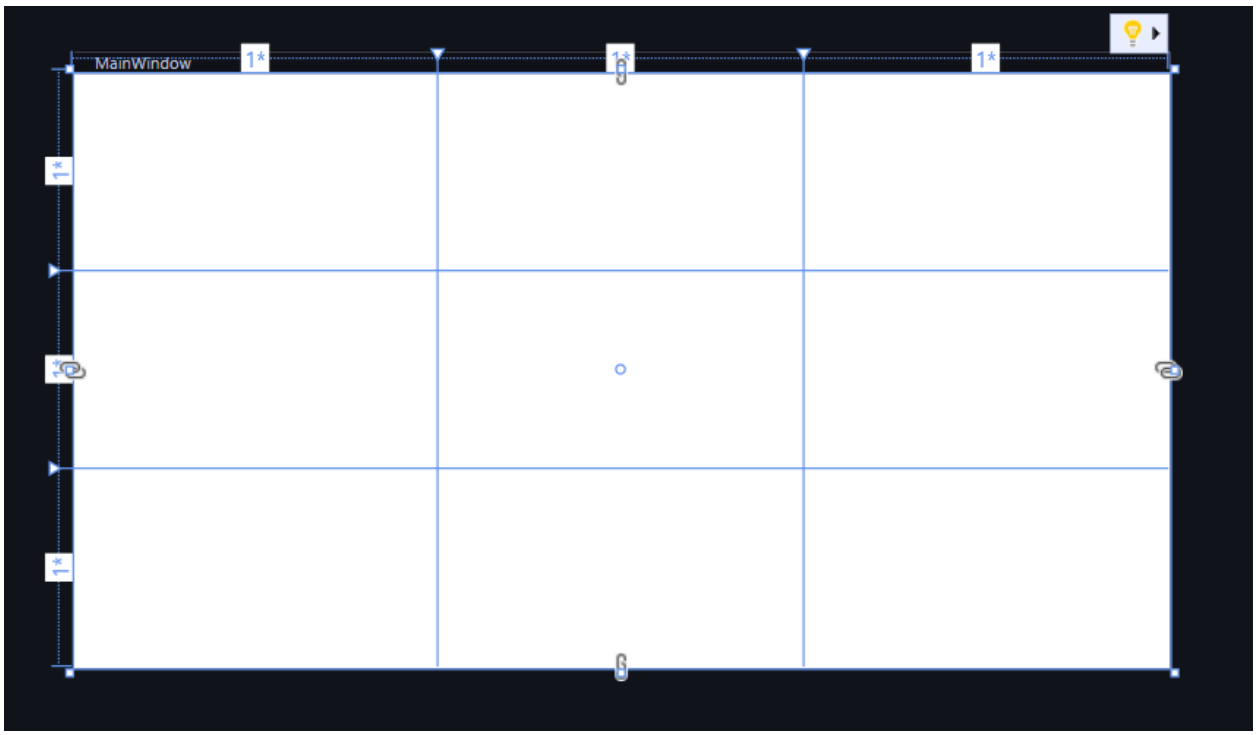
- является верхнеуровневым элементом интерфейса
- напрямую взаимодействует с операционной системой
- может существовать независимо от других окон
- может открывать другие окна

Grid

Это наиболее часто используемый контейнер, напоминающий обычную таблицу. Он содержит столбцы и строки, количество которых задает разработчик. Для определения строк используется свойство RowDefinitions, а для определения столбцов - свойство ColumnDefinitions.

```
<Grid.RowDefinitions>
   <RowDefinition></RowDefinition>
   <RowDefinition></RowDefinition>
   <RowDefinition></RowDefinition>
</Grid.RowDefinitions>
<Grid.ColumnDefinitions>
   <ColumnDefinition></ColumnDefinition>
   <ColumnDefinition></ColumnDefinition>
   <ColumnDefinition></ColumnDefinition>
</Grid.ColumnDefinitions>
```

Если вписать эти строчки в разметку, то мы сможем заметить, что наше окно разделилось на столбцы и строки

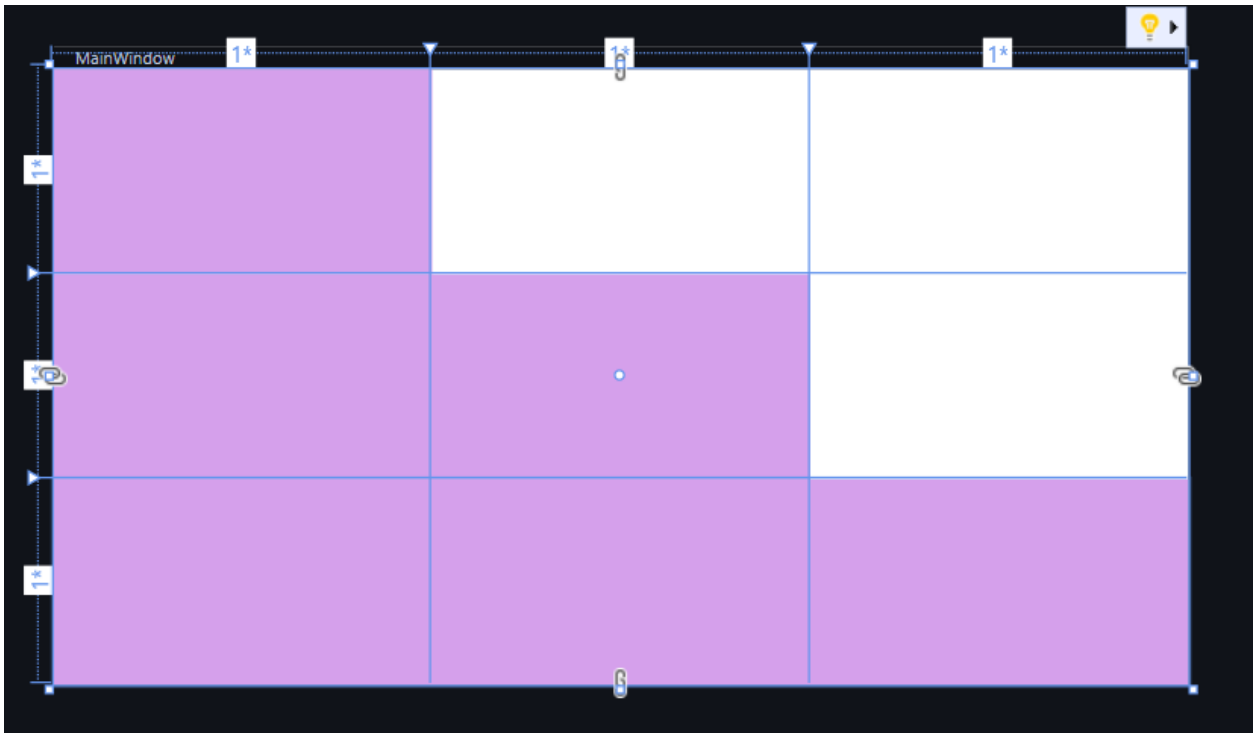


Поля видны только при разработке, при желании их можно отобразить и во время запуска приложения.

Чтобы задать позицию элемента управления с привязкой к определенной ячейке Grid, в разметке элемента нужно прописать значения свойств Grid.Column и Grid.Row, тем самым указывая, в каком столбце и строке будет находиться элемент. Кроме того, если мы хотим растянуть элемент управления на несколько строк или столбцов, то можно указать свойства Grid.ColumnSpan и Grid.RowSpan

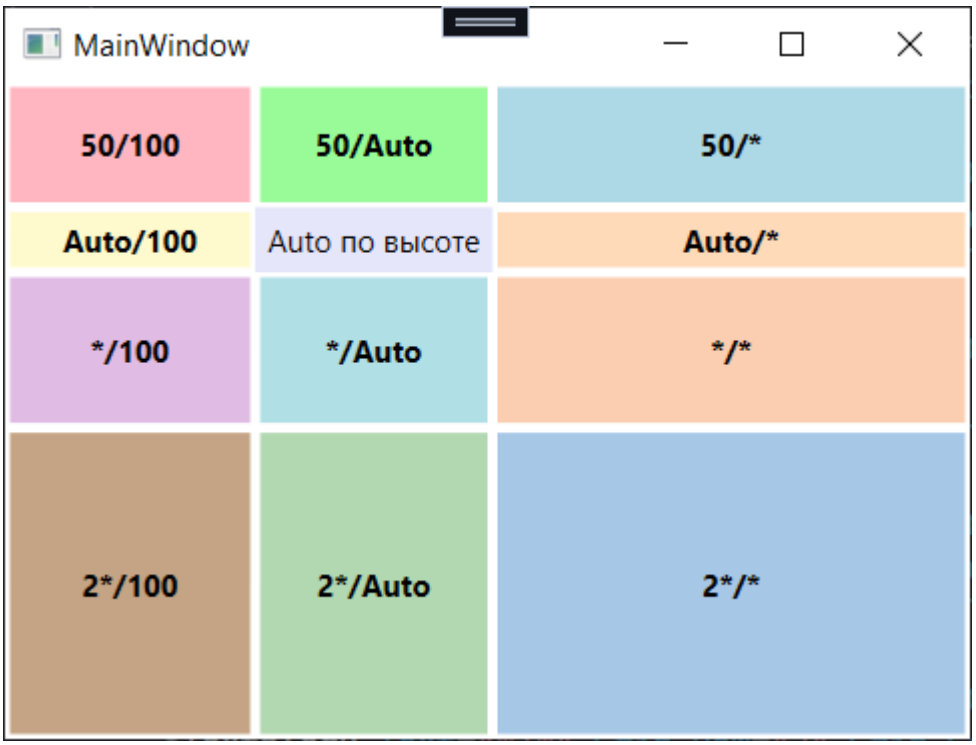
```
<Grid>
   <Grid.RowDefinitions>
      <RowDefinition></RowDefinition>
      <RowDefinition></RowDefinition>
      <RowDefinition></RowDefinition>
   </Grid.RowDefinitions>
   <Grid.ColumnDefinitions>
      <ColumnDefinition></ColumnDefinition>
      <ColumnDefinition></ColumnDefinition>
      <ColumnDefinition></ColumnDefinition>
   </Grid.ColumnDefinitions>
```

```
    <Rectangle Fill="#d5a0eb" Grid.Column="0" Grid.Row="0"/>
    <Rectangle Fill="#d5a0eb" Grid.ColumnSpan="2" Grid.Column="0" Grid.Row="1"/>
    <Rectangle Fill="#d5a0eb" Grid.Column="0" Grid.Row="3" Grid.ColumnSpan="3"/>
</Grid>
```



Для строк и столбцов можно устанавливать размеры: высоту для строк и ширину для столбцов. Можно указывать как абсолютные размеры, так и автоматические размеры с помощью ключевого слова «Auto», столбец или строка займет столько места, сколько им необходимо. Также можно устанавливать пропорциональные размеры с помощью символа «*», это значит, то элемент займет все оставшееся место, в отличие от auto, который занимает только необходимое ему место.

```
<Grid>
   <Grid.RowDefinitions>
      <RowDefinition Height="50"/>
      <RowDefinition Height="Auto"/>
      <RowDefinition Height="*"/>
      <RowDefinition Height="2*"/>
   </Grid.RowDefinitions>

   <Grid.ColumnDefinitions>
      <ColumnDefinition Width="100"/>
      <ColumnDefinition Width="Auto"/>
      <ColumnDefinition Width="*"/>
   </Grid.ColumnDefinitions>

   <Rectangle Fill="#FFB6C1" Grid.Row="0" Grid.Column="0" Margin="2"/>
   <Rectangle Fill="#98FB98" Grid.Row="0" Grid.Column="1" Margin="2"/>
   <Rectangle Fill="#ADD8E6" Grid.Row="0" Grid.Column="2" Margin="2"/>

   <Rectangle Fill="#FFFACD" Grid.Row="1" Grid.Column="0" Margin="2"/>
   <Rectangle Fill="#E6E6FA" Grid.Row="1" Grid.Column="1" Margin="2"/>
   <Rectangle Fill="#FFDAB9" Grid.Row="1" Grid.Column="2" Margin="2"/>

   <Rectangle Fill="#E0BBE4" Grid.Row="2" Grid.Column="0" Margin="2"/>
   <Rectangle Fill="#B0E0E6" Grid.Row="2" Grid.Column="1" Margin="2"/>
   <Rectangle Fill="#FBCEB1" Grid.Row="2" Grid.Column="2" Margin="2"/>

   <Rectangle Fill="#C4A484" Grid.Row="3" Grid.Column="0" Margin="2"/>
   <Rectangle Fill="#B2D8B2" Grid.Row="3" Grid.Column="1" Margin="2"/>
   <Rectangle Fill="#A7C7E7" Grid.Row="3" Grid.Column="2" Margin="2"/>
</Grid>
```

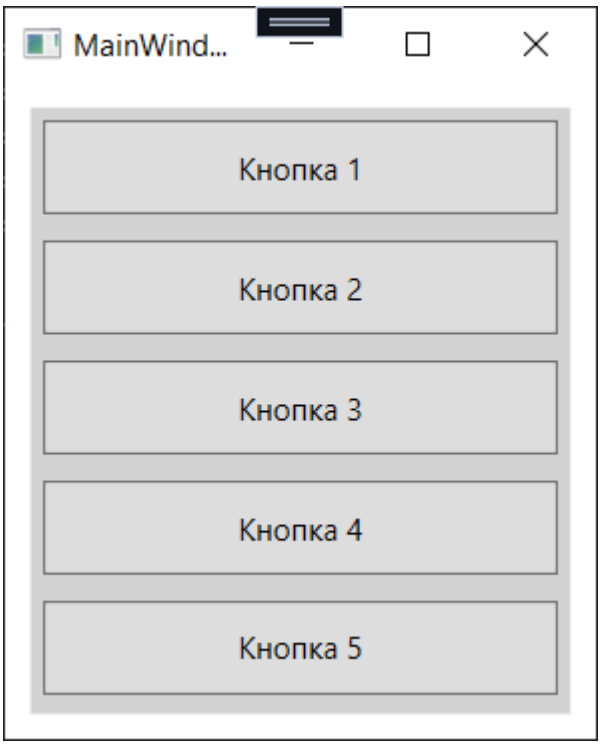


StackPanel

Если Grid позволяет строить сложные таблицы из строк и столбцов, то StackPanel решает другую задачу — последовательное размещение элементов.

StackPanel — это один из самых простых и интуитивно понятных контейнеров в WPF.

```
<StackPanel Background="LightGray" Margin="10">
   <Button Content="Кнопка 1" Margin="5" Padding="10"/>
   <Button Content="Кнопка 2" Margin="5" Padding="10"/>
   <Button Content="Кнопка 3" Margin="5" Padding="10"/>
   <Button Content="Кнопка 4" Margin="5" Padding="10"/>
   <Button Content="Кнопка 5" Margin="5" Padding="10"/>
   <TextBlock Text="Текстовый блок" Margin="5"
         HorizontalAlignment="Center" FontSize="16"/>
</StackPanel>
```



StackPanel — это контейнер компоновки, который размещает элементы интерфейса последовательно друг за другом либо по горизонтали, либо по вертикали в зависимости от ориентации (по умолчанию всегда вертикальная).

Для настройки ориентации используется свойство «Orientation», которая принимает параметры «Vertical» и «Horizontal»

```
<StackPanel Orientation="Horizontal">
```

```
    <StackPanel Orientation="Horizontal" Background="LightGray" Margin="10">
        <Button Content="Кнопка 1" Margin="5" Padding="10"/>
        <Button Content="Кнопка 2" Margin="5" Padding="10"/>
        <Button Content="Кнопка 3" Margin="5" Padding="10"/>
        <Button Content="Кнопка 4" Margin="5" Padding="10"/>
        <TextBlock Text="Текст" VerticalAlignment="Center" Margin="10"/>
    </StackPanel>
```

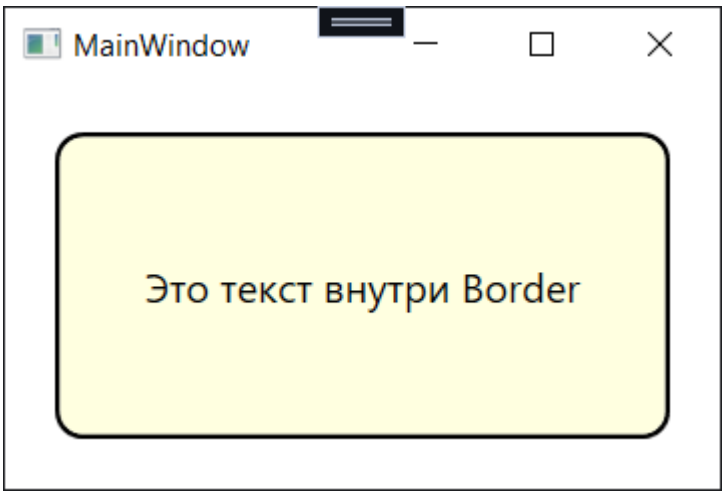
Border

Border — это элемент-декоратор, который предназначен для добавления визуального оформления вокруг другого элемента. Он не является контейнером компоновки в чистом виде (как Grid или StackPanel), но активно используется для улучшения внешнего вида интерфейса. Border может содержать только один дочерний элемент, поэтому если нужно оформить группу элементов, их сначала помещают в контейнер (StackPanel, Grid и т.д.), а затем этот контейнер оборачивают в Border.

Свойства:

- BorderBrush — цвет рамки
- BorderThickness — толщина рамки
- Background — фон
- CornerRadius — скругление углов

```
<Grid Margin="20">
  <Border BorderBrush="Black"
          BorderThickness="2"
          Background="LightYellow"
          Padding="20"
          CornerRadius="10">
    <TextBlock Text="Это текст внутри Border"
               FontSize="16"
               HorizontalAlignment="Center"
               VerticalAlignment="Center"/>
  </Border>
</Grid>
```



СВОЙСТВА КОМПОНОВКИ

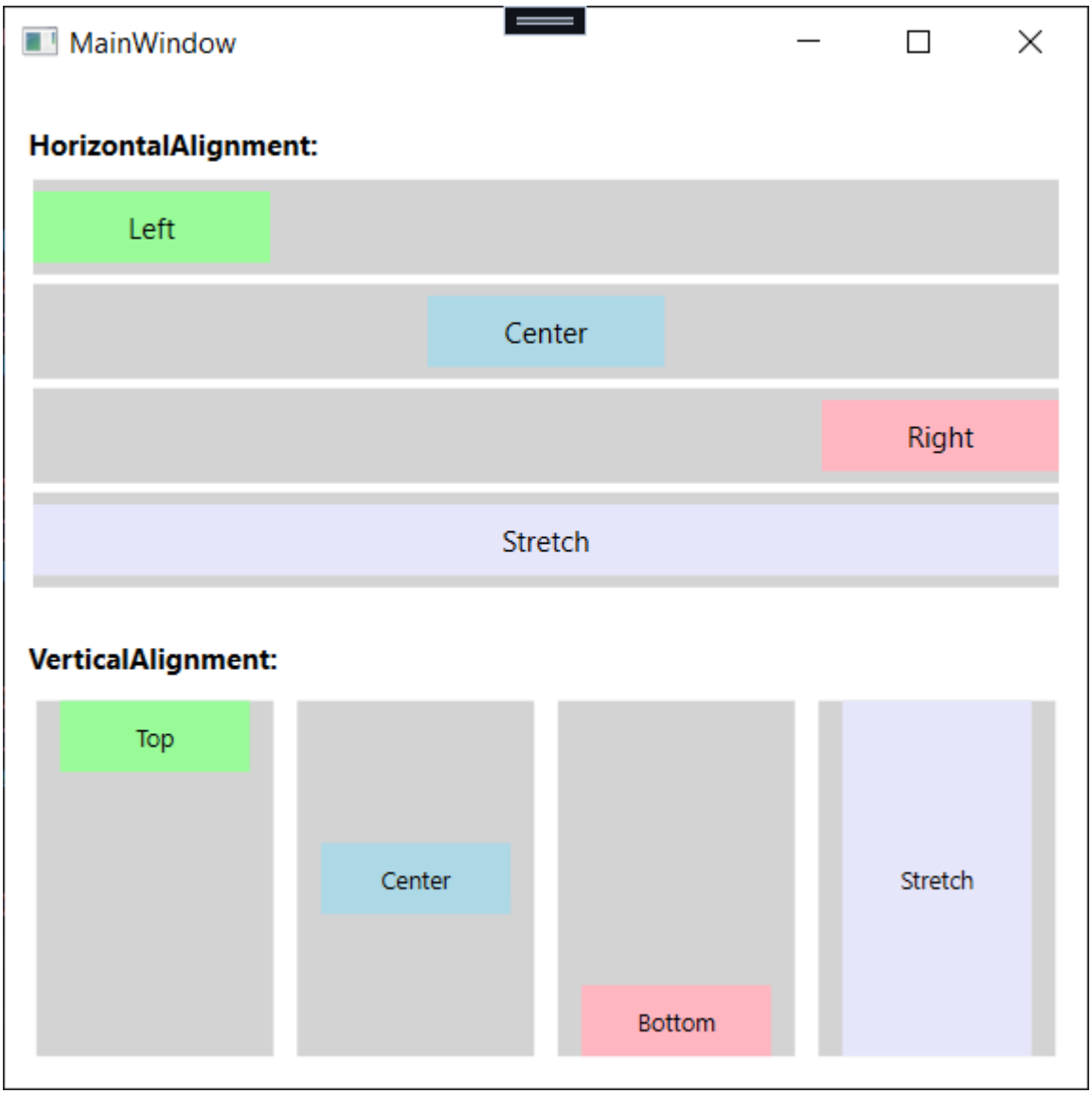
Ширина и высота (Width и Height)

Устанавливают ширину и высоту элемента, можно задать фиксированный, но нежелательно. Также можно установить максимально и минимально возможный диапазон с помощью MinWidth/MaxWidth и MinHeight/MaxHeight.

Выравнивание

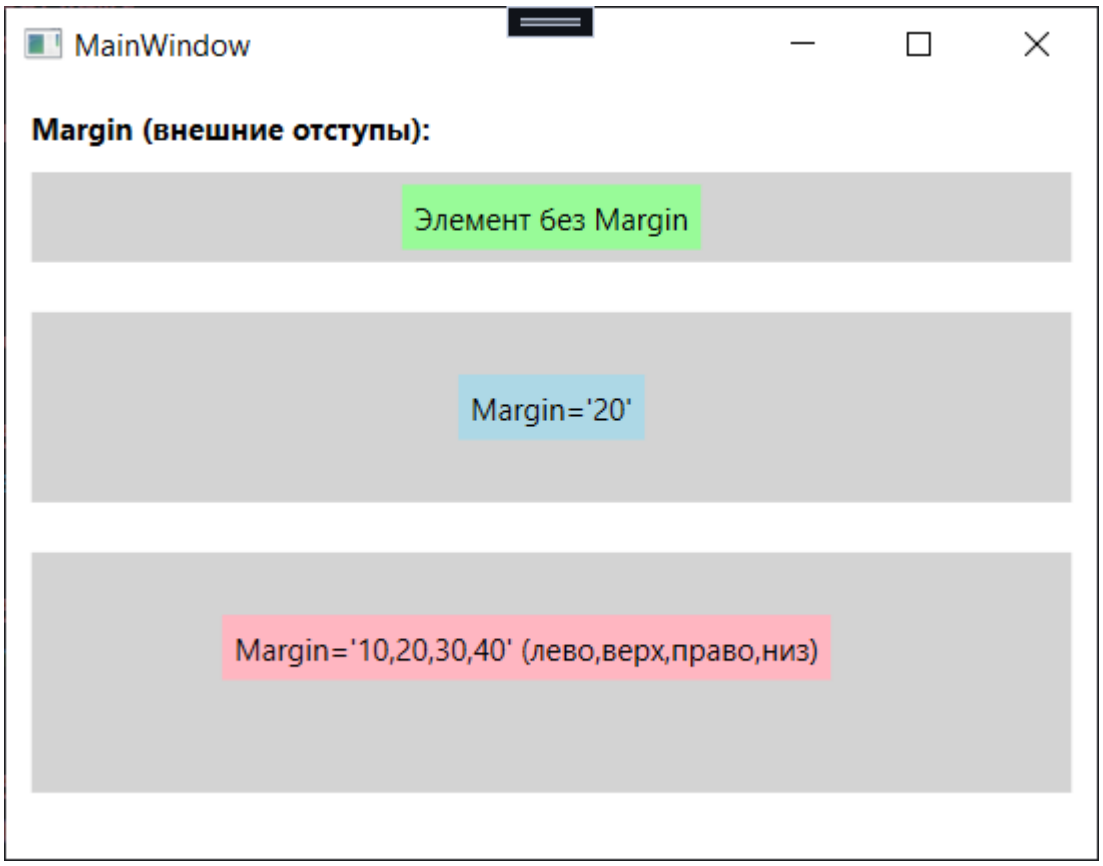
Свойство **HorizontalAlignment** выравнивает элемент по горизонтали относительно правой или левой стороны контейнера и соответственно может принимать значения Left, Right, Center (положение по центру), Stretch (растяжение по всей ширине).

Свойство **VerticalAlignment** выравнивает элемент по вертикали и принимает значения: Top (положение в верху контейнера), Bottom (положение внизу), Center (положение по центру), Stretch (растяжение по всей высоте).



Отступы

Свойство **Margin** устанавливает отступы вокруг элемента. Синтаксис: Margin="левый_отступ верхний_отступ правый_отступ нижний_отступ". Если установить одно значение, то оно применится для всех сторон.



ЭЛЕМЕНТЫ УПРАВЛЕНИЯ

Элементы управления (Controls) — это готовые визуальные компоненты интерфейса, предназначенные для отображения информации и взаимодействия пользователя с программой.

Все элементы управления могут быть условно разделены на несколько подгрупп:

- Элементы управления содержимым, например кнопки (Button), метки (Label)
- Специальные контейнеры, которые содержат другие элементы, но в отличие от элементов Grid или Canvas не являются контейнерами компоновки - ScrollViewer,GroupBox
- Декораторы, предназначение которых заключается в создание определенного фона вокруг вложенных элементов, например, Border или Viewbox.

– Элементы управления списками, например, ListBox, ComboBox.

– Текстовые элементы управления, например, TextBox, RichTextBox.

– Элементы, основанные на диапазонах значений, например, ProgressBar, Slider.

– Элементы для работ с датами, например, DatePicker и Calendar.

– Остальные элементы управления, которые не вошли в предыдущие подгруппы, например, Image.

У элементов управления так же есть ряд свойств:

Name (но чаще x:Name) присваивает имя элементу, в последствии можно обращаться к нему через код.

Visibility устанавливает видимость элемента и принимает значения:

– Visible - элемент виден и участвует в компоновке.

– Collapsed - элемент не виден и не участвует в компоновке.

– Hidden - элемент не виден, но при этом участвует в компоновке.

Cursor устанавливает курсор, например, при наведении на элемент

Background задает цвет фона, можно выбрать из предложенных, либо присвоить свой цвет в кодировке hex (#fffff)

Foreground имеет те же самые характеристики, но присваивает цвет шрифту

Content позволяет поместить текст в элемент (для кнопок и Label)

Text позволяет поместить текст в элемент TextBlock

Свойства шрифтов:

– FontFamily - определяет семейство шрифта (например, Arial, Verdana и т.д.)

– FontSize - определяет высоту шрифта

– FontStyle - определяет наклон шрифта, принимает одно из трех значений - Normal, Italic,Oblique.

– FontWeight - определяет толщину шрифта и принимает ряд значений, как Black,Bold и др.

Кнопки

Одним из самых часто используемых элементов управления в WPF является кнопка (Button).

Button — это элемент управления, предназначенный для выполнения действия при нажатии пользователем. Выполнение действий осуществляется через обработчик событий.

Обработчик события — это метод программы, который выполняется в ответ на событие, то есть на действие.

Для кнопки обработчик события создается с помощью свойства «Click», его можно либо прописать в разметке, либо дважды кликнуть на кнопку.

```
<Button Content="Обычная кнопка"
        Margin="5" Padding="10"
        Background="LightBlue"
        Foreground="Black"
        FontSize="14"/>
```

Текст

Для вывода текста можно использовать **TextBlock**.

```
<TextBlock Text="TextBlock - для отображения текста"
  FontSize="18" FontWeight="Bold" Margin="0,0,0,20"/>
```

Для изменения параметров отображаемого текста данный элемент имеет такие свойства, как LineHeight, TextWrapping и TextAlignment.

Свойство **LineHeight** позволяет указывать высоту строк.

Свойство **TextWrapping** позволяет переносить текст при установке этого свойства TextWrapping="Wrap". По умолчанию это свойство имеет значение NoWrap, поэтому текст не переносится.

Свойство **TextAlignment** выравнивает текст по центру (значение Center), правому (Right) или левому краю (Left)

Для ввода текста используется **TextBox.**

Так же, как и TextBlock, имеет свойства TextWrapping и TextAlignment

С помощью свойства **MaxLength** можно задать предельное количество вводимых символов.

```
<TextBox Grid.Row="5" Margin="5" Padding="5" FontSize="14"/>
```

Для ввода текста, а точнее паролей правильнее будет использовать **PasswordBox**, в отличии от TextBox здесь используется маска, которая скрывает введенный текст, с помощью PasswordChar можно установить символ, которым будут заменяться символы. По умолчанию устанавливается •.

```
<PasswordBox Grid.Row="8" Margin="5" Padding="5" PasswordChar="*"/>
```

Прокрутка

Элемент **ScrollViewer** обеспечивает прокрутку содержимого. Может вмещать в себя только один элемент, поэтому все элементы, помещаемые внутрь ScrollViewer необходимо облачить в еще один контейнер. Например, прописать внутри Grid или StackPanel.

ScrollViewer содержит в себе 2 элемента HorizontalScrollBarVisibility и VerticalScrollBarVisibility, отвечающие за горизонтальную и вертикальную прокрутку соответственно. Они принимают следующие значения:

– Auto: наличие полос прокрутки устанавливается автоматически

– Visible: полосы прокрутки отображаются в окне приложения

– Hidden: полосы прокрутки не видно, но прокрутка возможна с помощью клавиш клавиатуры

– Disabled: полосы прокрутки не используются, а сама прокрутка даже с помощью клавиатуры невозможна

Списки

При создании приложений практически всегда есть потребность в отображении набора данных, а не только одного объекта, один из примеров — список товаров. Для облегчения реализации такого функционала существуют элементы управления списками, которые хранят коллекции, отображают все объекты (в рамках коллекции), работают с привязкой данных. Основные элементы, которые нам понадобятся — ListBox и ComboBox.

Элементы списков:

1. SelectedItem — выбранный объект. Позволяет производить действия с выбранным объектом

2. SelectedIndex — индекс выбранного элемента. Чаще применяется для ComboBox

3. DisplayMemberPath — указывает, что необходимо вывести. Например, название товара.

ListBox

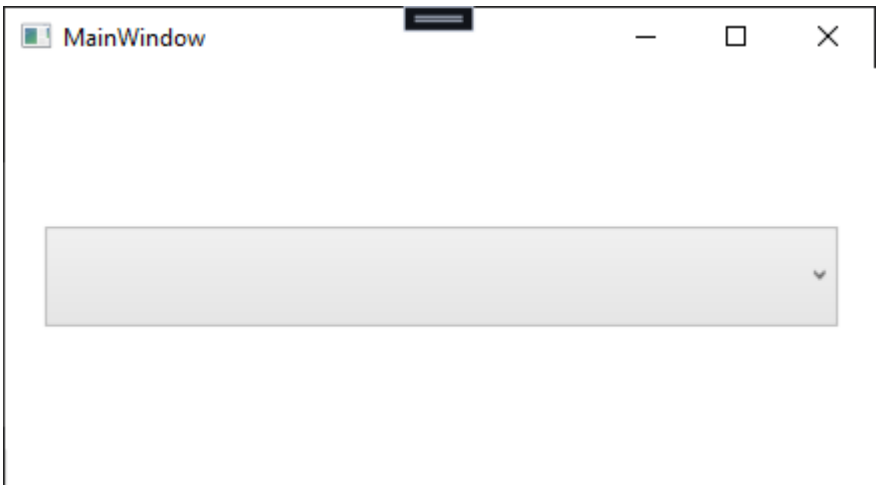
ListBox — это элемент управления, который отображает список элементов вертикально (или горизонтально) и позволяет выбирать один или несколько элементов.

```
<ListBox>
   <ListBoxItem>Товар 1</ListBoxItem>

   // или <ListBoxItem Content="Товар 1"/>

   <ListBoxItem>Товар 2</ListBoxItem>
</ListBox>
```

ComboBox

ComboBox представляет собой раскрывающийся список.

```
<ComboBox>
   <ComboBoxItem>Категория 1</ComboBoxItem>
   <ComboBoxItem>Категория 2</ComboBoxItem>
</ComboBox>
```



Привязка данных

Привязка подразумевает взаимодействие двух объектов: источника и приемника. Объект-приемник создает привязку к определенному свойству объекта-источника. В случае модификации объекта-источника, объект-приемник также будет модифицирован.

Когда мы пишем Binding, мы фактически говорим системе: «Вот это свойство элемента интерфейса должно отображать вот это свойство объекта». Например, TextBlock должен показывать Name клиента. Или ComboBox должен показывать список тренеров. И дальше WPF сам следит за изменениями. Если данные поменялись — интерфейс обновился. Если пользователь что-то ввёл — данные могут обновиться обратно.

DataContext — это как текущая «рабочая область данных». Это объект, внутри которого Binding ищет свойства по умолчанию. Можно представить, что DataContext — это как папка на компьютере. Если вы уже открыли папку, вам не нужно писать полный путь к файлу, достаточно написать имя файла. Точно так же, если DataContext установлен в объект Client, то Binding {Name} автоматически означает Client.Name.

Очень важно понимать, что DataContext наследуется. Если вы установили DataContext у Grid, то все элементы внутри Grid автоматически получают тот же источник данных. Благодаря этому код становится намного чище и проще.

ItemsSource — это способ сказать элементу списка: «Вот коллекция объектов, которые ты должен показать». Не строки, не текст, не просто значения — именно объекты. Например, список клиентов. Каждый клиент — это отдельный объект. И WPF сам создаёт визуальный элемент для каждого объекта в коллекции.

DataTemplate — это описание внешнего вида одного объекта данных. Это как форма, по которой WPF «рисует» каждый элемент списка. Сам объект — это только данные. У него нет внешнего вида. Внешний вид задаёт именно DataTemplate.

Например, объект Client содержит Name, Phone, Membership. Но DataTemplate решает, что Name будет крупным текстом, Phone — серым мелким текстом, а абонемент — справа. И если мы захотим поменять внешний вид — нам не нужно менять класс Client. Мы просто меняем DataTemplate.

Стили

Стили позволяют определить набор некоторых свойств и их значений, которые потом могут применяться к элементам в xaml. Стили хранятся в ресурсах и отделяют значения свойств элементов от пользовательского интерфейса. Также стили могут задавать некоторые аспекты поведения элементов с помощью триггеров. Аналогом стилей могут служить каскадные таблицы стилей (CSS), которые применяются в коде html на веб-страницах.

Если объяснить максимально жизненно, стиль — это набор правил внешнего вида и поведения, который можно применить сразу к множеству элементов. Это как шаблон оформления.

В WPF **стиль** — это объект, который содержит набор свойств, которые нужно применить к элементу управления. Например: цвет фона, цвет текста, шрифт, толщина рамки, отступы, а иногда даже поведение.

Когда мы используем стили, мы решаем сразу несколько проблем. Во-первых, убираем дублирование кода. Во-вторых, получаем единый внешний вид приложения. В-третьих, можем менять внешний вид централизованно — поменяли стиль, и изменились сразу все элементы.

Теперь важно понять, где вообще живут стили. В WPF они чаще всего хранятся в ресурсах. **Ресурсы** — это как словарь, в котором лежат объекты, которые можно переиспользовать: цвета, кисти, стили, шаблоны.

WPF поддерживает два типа ресурсов:

StaticResource — ресурс загружается один раз при создании элемента

DynamicResource — ресурс может измениться во время выполнения

И вот здесь появляется очень важная идея: стиль можно объявить один раз, а использовать в сотнях элементов.

Стиль создается как ресурс с помощью объекта **Style**, который представляет класс System.Windows.Style. И как любой другой ресурс, он обязательно должен иметь ключ.

С помощью коллекции **Setters** определяется группа свойств, входящих в стиль. В нее входят объекты Setter, которые имеют следующие свойства:

Property

Указывает на свойство, к которому будет применяться данный сеттер. Имеет следующий синтаксис

```
Property="Тип_элемента.Свойство_элемента".
```

Выше в качестве типа элемента использовался Control, как общий для всех элементов. Поэтому данный стиль мы могли бы применить и к Button, и к TextBlock, и к другим элементам.

Value

Устанавливает значение.

Если значение свойства представляет сложный объект, то мы можем его вынести в отдельный элемент

Нам необязательно прописывать для всех кнопок стиль. Мы можем в самом определении стиля с помощью свойства TargetType задать тип элементов. В этом случае стиль будет автоматически применяться ко всем кнопкам в окне.

x:Key

Уникальный идентификатор стиля, по которому мы будем к нему обращаться

TargetType

Тип элементов, к которым может применяться стиль (Button, TextBlock, TextBox и т.д.).

Если не указывать x:Key, а задать только TargetType, стиль применится ко всем элементам этого типа.

BasedOn

Стили, как и классы можно наследовать, производный стиль будет наследовать все свойства базового стиля, а также может добавлять и изменять свои свойства.

Базовый стиль для кнопок

```
<Style TargetType="Button">
    <Setter Property="FontSize" Value="14"/>
    <Setter Property="Padding" Value="15,8"/>
    <Setter Property="Margin" Value="5"/>
    <Setter Property="Cursor" Value="Hand"/>
    <Setter Property="Height" Value="40"/>
</Style>
```

Наследование стиля

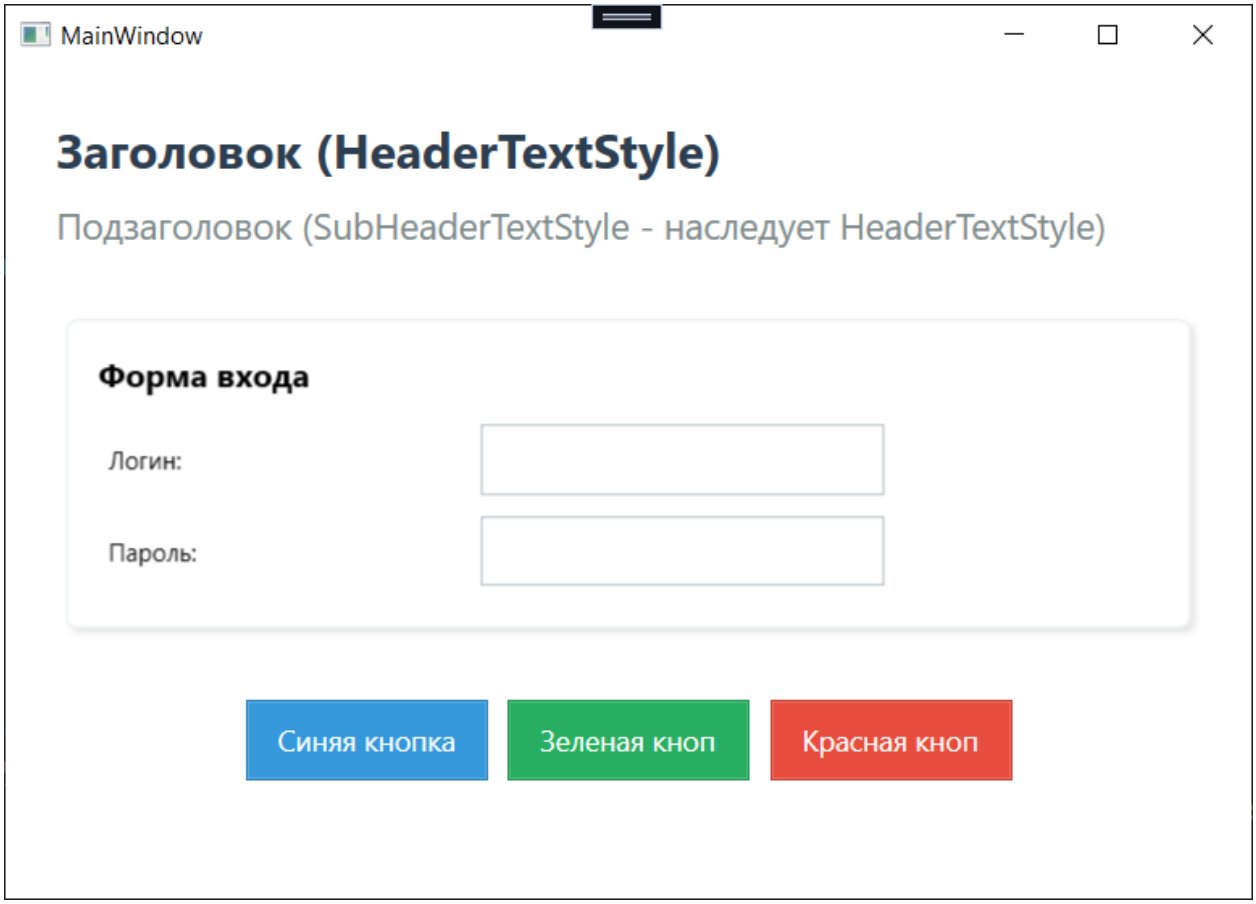
```
<Style x:Key="BlueButtonStyle" TargetType="Button" BasedOn="{StaticResource {x:Type Button}}">
    <Setter Property="Background" Value="#3498db"/>
    <Setter Property="Foreground" Value="White"/>
    <Setter Property="BorderBrush" Value="#2980b9"/>
    <Setter Property="BorderThickness" Value="1"/>
    <Style.Triggers>
      <Trigger Property="IsMouseOver" Value="True">
        <Setter Property="Background" Value="#2980b9"/>
      </Trigger>
      <Trigger Property="IsPressed" Value="True">
        <Setter Property="Background" Value="#1f618d"/>
      </Trigger>
    </Style.Triggers>
</Style>
```

Стиль для текста

```
<Style x:Key="HeaderTextStyle" TargetType="TextBlock">
    <Setter Property="FontSize" Value="24"/>
    <Setter Property="FontWeight" Value="Bold"/>
    <Setter Property="Foreground" Value="#2c3e50"/>
    <Setter Property="Margin" Value="5"/>
</Style>
```

Пример использования

```
<TextBlock Text="Подзаголовок (SubHeaderTextStyle - наследует HeaderTextStyle)" Style="{StaticResource SubHeaderTextStyle}"/>

<Button Content="Синяя кнопка" Style="{StaticResource BlueButtonStyle}" Width="120"/>
```



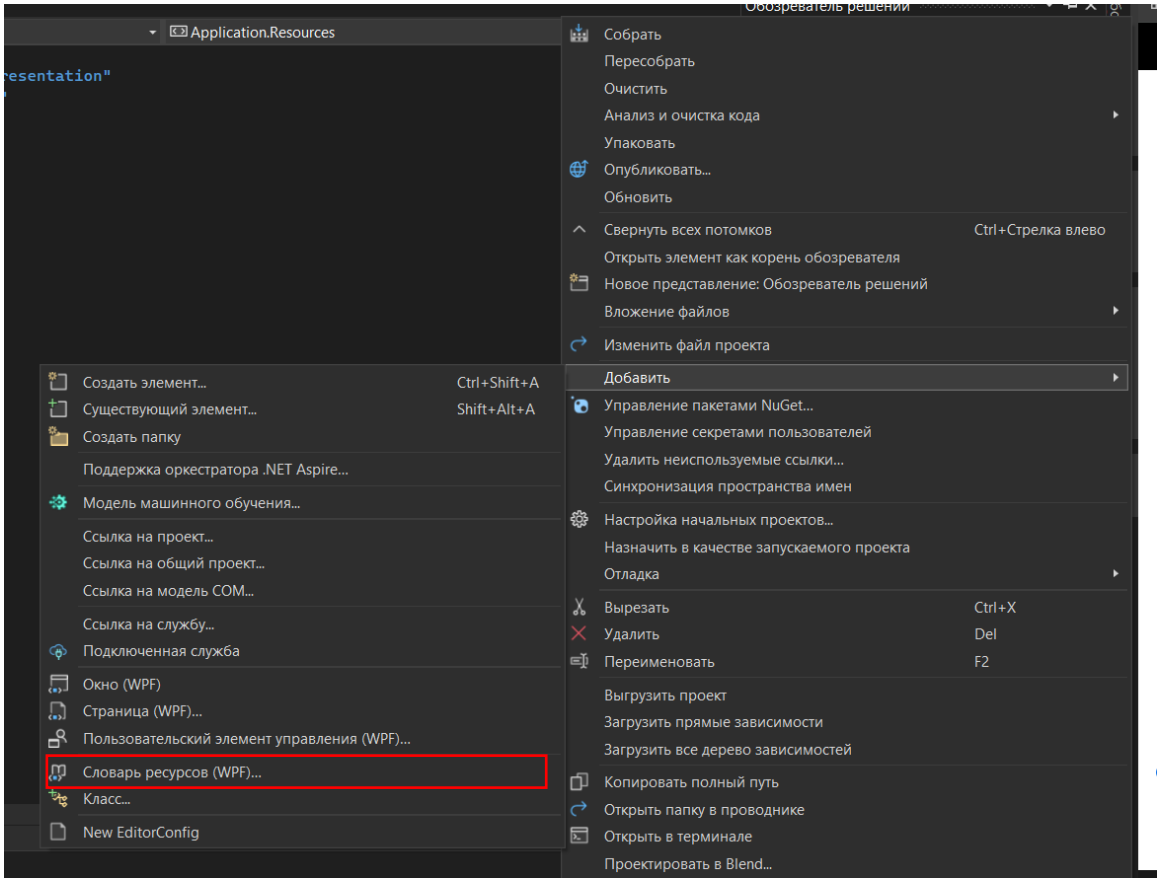
Стили можно прописать в файлике App.xaml

Изначально он выглядит так:

```
<Application x:Class="StylesRes.App"
         xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
         xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
         xmlns:local="clr-namespace:StylesRes"
         StartupUri="MainWindow.xaml">
   <Application.Resources>
   </Application.Resources>
</Application>
```

Можно также создать словарь ресурсов и ссылаться на него в App.xaml

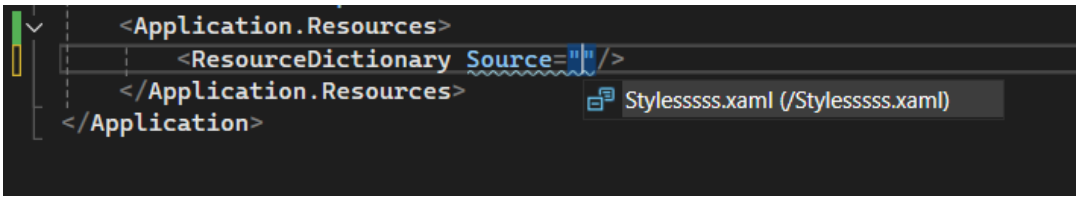
Для этого ПКМ по проекту



В App.xaml пропишем

```
<Application.Resources>
   <ResourceDictionary Source="/Stylesssss.xaml"/>
</Application.Resources>
```

Visual Studio предложит выбрать словарь ресурсов



Цвет

Цвет — это фундаментальная часть любого пользовательского интерфейса. В WPF работа с цветами реализована через структуру Color из

пространства имен System.Windows.Media. Цвета используются везде: для фона элементов (Background), для текста (Foreground), для рамок (BorderBrush), для заливки фигур (Fill) и многих других свойств.

В XAML существует несколько способов указать цвет:

1. Стандартные именованные цвета

WPF предоставляет более 140 предопределенных цветов с понятными именами. Они определены в классе Colors и могут использоваться напрямую в XAML.

```
<Button Background="Red" Content="Красная кнопка"/>
<TextBlock Foreground="Blue" Text="Синий текст"/>
<Border BorderBrush="Green" BorderThickness="2"/>
```

2. HEX-кодировка

HEX-кодировка — это компактный способ задания цвета в шестнадцатеричной системе. Формат: #AARRGGBB или #RRGGBB.

#RRGGBB — без прозрачности (альфа-канал считается FF — полностью непрозрачный)

#AARRGGBB — с прозрачностью (AA — альфа-канал)

Каждая пара символов — это значение от 00 до FF в шестнадцатеричной системе (0-255 в десятичной).

Можно также управлять прозрачностью цвета, приписав код вначале обозначения цвета

Таблица прозрачности

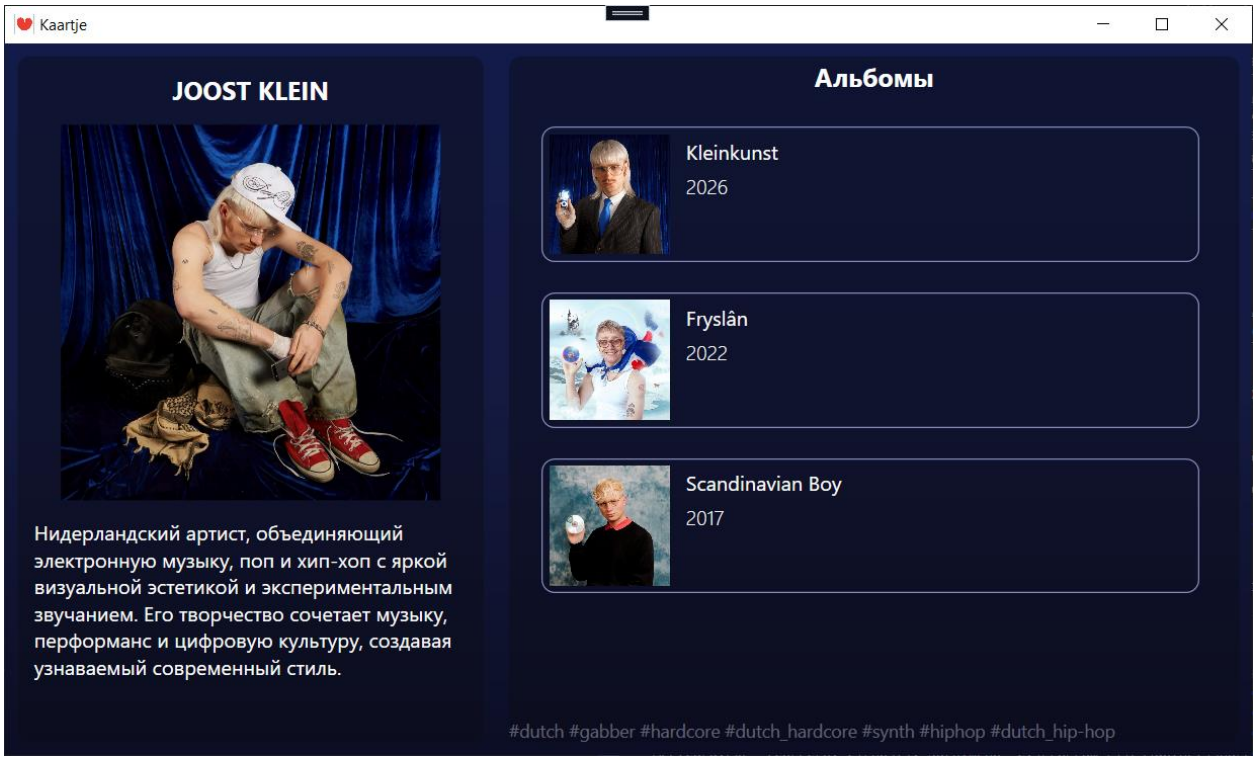
Прозрачность	**HEX**	**Пример**
10%	E6	#E6FF0000
20%	CC	#CCFF0000
30%	B3	#B3FF0000
40%	99	#99FF0000
50%	80	#80FF0000
60%	66	#66FF0000
70%	4D	#4DFF0000
80%	33	#33FF0000
90%	1A	#1AFF0000

```
        <ScrollViewer VerticalScrollBarVisibility="Auto">
            <StackPanel Margin="20">


                <TextBlock Text="От непрозрачного к прозрачному:"
FontWeight="Bold" Margin="0,10,0,10"/>
                <StackPanel>
                    <StackPanel Orientation="Horizontal"
HorizontalAlignment="Center">
                        <Rectangle Fill="#FFFF0000" Width="80" Height="80"
Margin="5"/>
                        <Rectangle Fill="#C0FF0000" Width="80" Height="80"
Margin="5"/>
                        <Rectangle Fill="#80FF0000" Width="80" Height="80"
Margin="5"/>
                        <Rectangle Fill="#40FF0000" Width="80" Height="80"
Margin="5"/>
                        <Border BorderBrush="Black" BorderThickness="0.5"
Margin="5">
                            <Rectangle Fill="#00FF0000" Width="80" Height="80" />
                        </Border>

                    </StackPanel>

                    <StackPanel Orientation="Horizontal"
HorizontalAlignment="Center" Margin="0,5,0,20">
                        <TextBlock Text="#FF0000" Width="80"
TextAlignment="Center" Margin="5"/>
                        <TextBlock Text="#C00000" Width="80"
TextAlignment="Center" Margin="5"/>
                        <TextBlock Text="#800000" Width="80"
TextAlignment="Center" Margin="5"/>
                        <TextBlock Text="#400000" Width="80"
TextAlignment="Center" Margin="5"/>
                        <TextBlock Text="#000000" Width="80"
TextAlignment="Center" Margin="5"/>
                    </StackPanel>
                </StackPanel>
```

```
            <TextBlock Text="Наложение цветов:" FontWeight="Bold"
Margin="0,20,0,10"/>
            <Grid HorizontalAlignment="Center">
              <Rectangle Fill="#FF00FF00" Width="150" Height="150"
Margin="5"/>

              <Rectangle Fill="#80FF0000" Width="100" Height="100"
Margin="30,30,0,0"/>

              <Rectangle Fill="#800000FF" Width="70" Height="70"
Margin="60,60,0,0"/>

            </Grid>

            <TextBlock Text="Зеленый (#FF00FF00) + Красный (#80FF0000) +
Синий (#800000FF)"
                    HorizontalAlignment="Center" Margin="0,10,0,0"/>
          </StackPanel>
        </ScrollViewer>
```

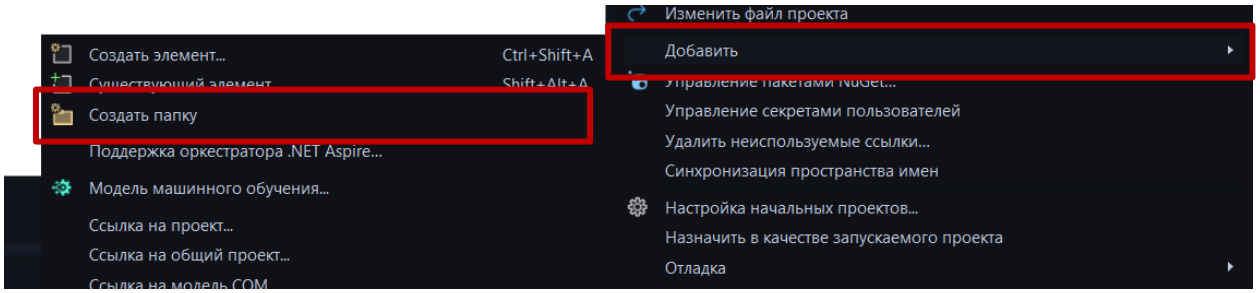
ПРИМЕР

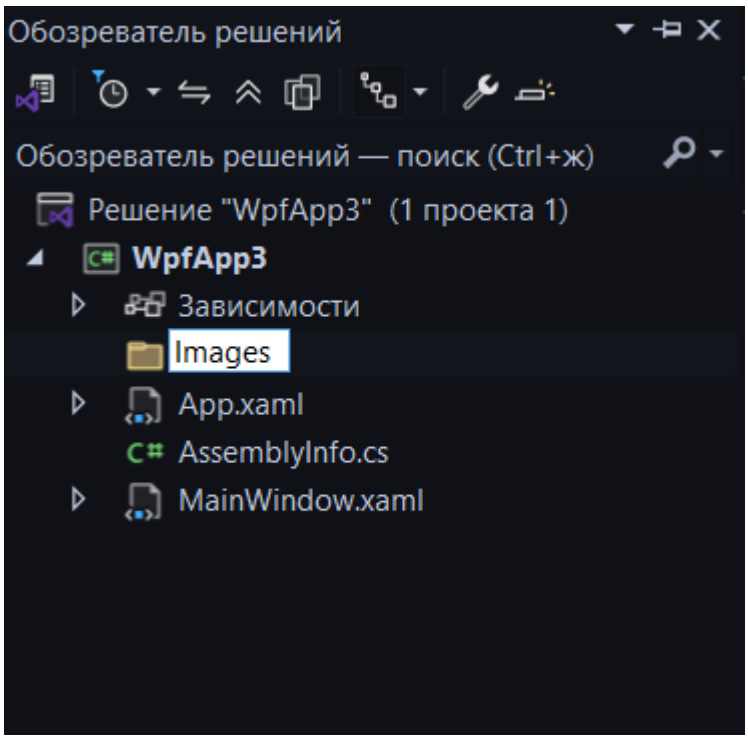
Пропишем и разберем разметку



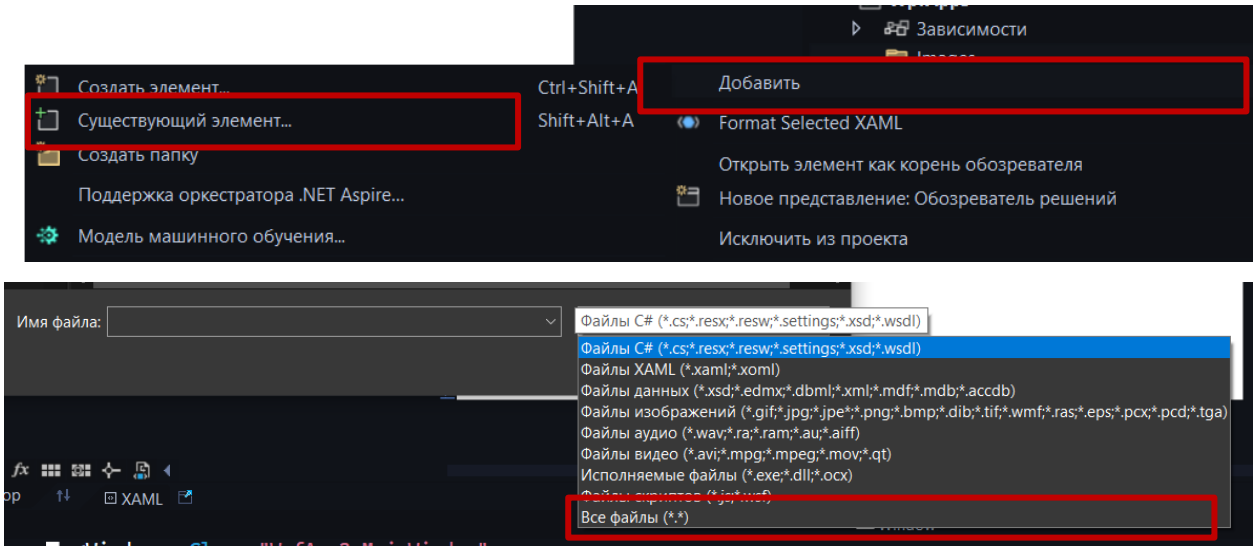
Сначала импортируем все необходимые ресурсы

В обозревателе решений нажмите правой кнопкой (ПКМ) мыши по проекту и в появившемся окне выберите добавить → Папка. Назовите ее Resources или Images.



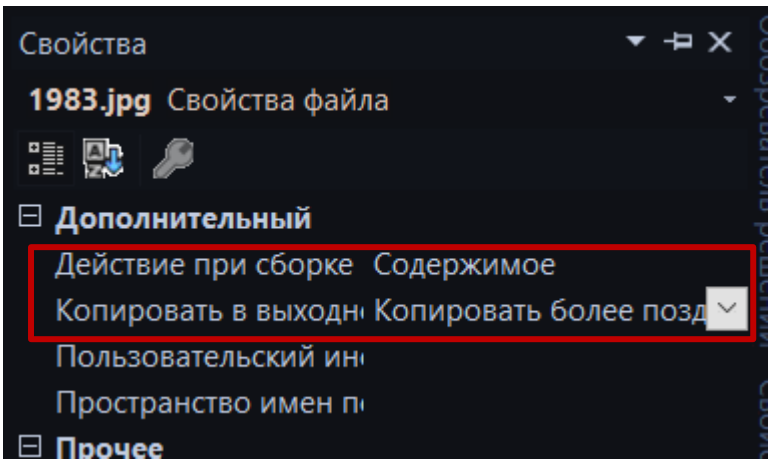


Нажмите ПКМ по папке → добавить → существующий элемент. В фильтрах установите «Все файлы»



Загрузим все необходимые изображения.

Выделим все изображения, далее перейдем в свойства и установим следующие значения



Теперь зададим базовые настройки окна:

```
Title="Kaartje" // заголовок окна
Width="1000" // ширина окна
Height="600" // высота окна
Icon="/Images/frys.png" // иконка окна
WindowStartupLocation="CenterScreen" // расположение окна при запуске
```

В исходном Grid пропишем фон. В примере используется градиент, но можно также использовать сплошную заливку (<Grid.Background=”цвет”>)

Для градиента

```
<Grid.Background>
   <LinearGradientBrush StartPoint="0,0" EndPoint="0,1"> // Объявление и направление градиента
      <GradientStop Offset="0" Color="#131b4a" /> // цвета и их положения
      <GradientStop Offset="0.5" Color="#141a40" />
      <GradientStop Offset="1" Color="#0b0e24" />
   </LinearGradientBrush>
</Grid.Background>
```

Разделим окно на 2 столбца

```
<Grid.ColumnDefinitions>
   <ColumnDefinition Width="1.3*" />
   <ColumnDefinition Width="2*" />
</Grid.ColumnDefinitions>
```

Первый столбец будет занимать информация о музыканте, второй столбец — его дискография.

В первом столбце разместим Border, но перед этим пропишем стили

```
<Style TargetType="Border" x:Key="BorderStyle">// стиль для Border
   <Setter Property="BorderThickness" Value="0"/> // Толщина границ 0
   <Setter Property="CornerRadius" Value="10"/> // Закругление углов
   <Setter Property="Margin" Value="10"/> // Отступы
   <Setter Property="Background" Value="#730a0c17"/> // Цвет Border. Здесь также устанавливается прозрачность цвета путем прописывания двух значений вначале
</Style>
```

Также пропишем стили для TextBlock

```
<Style TargetType="TextBlock" x:Key="MainTbStyle">// Стиль для основного текста
   <Setter Property="Foreground" Value="White"/> // Цвет шрифта
   <Setter Property="FontSize" Value="16"/> // Размер шрифта
   <Setter Property="Margin" Value="3"/> // Отступы
</Style>

<Style TargetType="TextBlock" BasedOn="{StaticResource MainTbStyle}" x:Key="HeaderStyle">// Стиль для заголовка (наследуется от главного)
   <Setter Property="FontWeight" Value="Bold"/> // Полужирный шрифт
   <Setter Property="FontSize" Value="20"/>
   <Setter Property="HorizontalAlignment" Value="Center"/> // Выравнивание по горизонтали
</Style>
```

Реализуем первый столбец

```
<Border Style="{StaticResource BorderStyle}">
   <Grid Margin="10">  // Border принимает только один элемент, поэтому пропишем контейнер
      <Grid.RowDefinitions> // Деление на строки
         <RowDefinition Height="auto" />
         <RowDefinition Height="*" />
      </Grid.RowDefinitions>

      <StackPanel> // Имя и фотография
         <TextBlock Style="{StaticResource HeaderStyle}" Text="JOOST KLEIN" />
```

```
                <Image Width="300" Height="300" Margin="10"
                    Source="/Images/joostAv.png" />
            </StackPanel>

            <TextBlock
                Grid.Row="1"
                Style="{StaticResource MainTbStyle}"
                TextWrapping="Wrap"> // Описание
                Нидерландский артист …
            </TextBlock>
        </Grid>
    </Border>
```

Реализуем второй столбец

```
    <Border Grid.Column="1" Style="{StaticResource BorderStyle}">
        <Grid>
            <Grid.RowDefinitions>
                <RowDefinition Height="auto" /> // строка для заголовка
                <RowDefinition Height="*" /> // строка для списка
                <RowDefinition Height="auto" /> // строка для тегов
            </Grid.RowDefinitions>

            <TextBlock  Style="{StaticResource  HeaderStyle}"  Text="Альбомы"
/>
            <ListBox
                Grid.Row="1" // работаем во втором столбце
                Margin="10"
                HorizontalAlignment="Stretch"
                Background="Transparent"
                BorderThickness="0"> // список для вывода альбомов
                <ListBoxItem> // элемент списка
                    <Border  Width="520"  Style="{StaticResource  AlbBorder}">  //
рамка для элемента
                        <Grid>
                            <Grid.ColumnDefinitions>
                                <ColumnDefinition Width="auto" />
                                <ColumnDefinition Width="*" />
                            </Grid.ColumnDefinitions>
```

```
                        <Image
                            Width="95"
                            Height="95"
                            Margin="5"
                            Source="/Images/kleinkunst.png" />

                        <StackPanel Grid.Column="1" Margin="5">
                            <TextBlock      Style="{StaticResource      MainTbStyle}"
Text="Kleinkunst" />
                            <TextBlock          Style="{StaticResource          YearTb}"
Text="2026" />
                        </StackPanel>
                    </Grid>
                </Border>
            </ListBoxItem>
        … Остальные элементы списка
                    </Grid>
                </Border>
            </ListBoxItem>

            </ListBox>

            <TextBlock     Grid.Row="2"     Text="#dutch     #gabber     #hardcore
#dutch_hardcore #synth #hiphop #dutch_hip-hop"
                    Style="{StaticResource TagsTb}"
                    TextWrapping="Wrap"/> // Теги исполнителя
        </Grid>
    </Border>
```